

PANDUAN LENGKAP & CHEAT SHEET DEMO REST API CRAFTIVE (EDISI SPESIAL & SANGAT DETAIL)

Mata Kuliah: Pemrograman API — D4 Manajemen Informatika, Fakultas Vokasi, Universitas Negeri Surabaya (UNESA) 2026

Dosen Pengampu: M Adamu Islam Mashuri, S.Tr.T., M.Tr.Kom (Pak Adam)

Mahasiswa: Selvi Adinda H. (NIM: 24091397145)

BAGIAN 1: Panduan Instalasi & Setup Proyek (Untuk Demo Laptop Baru / Penguji)

Jika Pak Adam meminta Anda menunjukkan cara menjalankan proyek dari nol di komputernya atau laptop lain, ikuti langkah berikut:

Langkah-Langkah Setup Singkat:

1. **Ekstrak atau Clone Folder Proyek** ke direktori server lokal Anda (misalnya `C:/xampp1/htdocs/craftive` atau `C:/xampp/htdocs/craftive`).

2. **Instalasi Dependencies (Backend & Frontend):**

- Buka terminal di root direktori proyek, jalankan:

```
composer install
npm install
```

3. **Konfigurasi File Environment (`.env`):**

- Salin file `.env.example` menjadi `.env`.
- Sesuaikan konfigurasi database MySQL Anda:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=craftive
DB_USERNAME=root
DB_PASSWORD=
```

4. **Generate Kunci Aplikasi & JWT Secret:**

- Jalankan perintah berikut untuk mengamankan enkripsi session dan token JWT:

```
php artisan key:generate
php artisan jwt:secret
```

5. **Migrasi Database & Seeding Data Awal:**

- Jalankan migrasi untuk membuat seluruh tabel dan mengisinya dengan akun uji coba awal:

```
php artisan migrate --seed
```

6. Jalankan Server Lokal:

- Gunakan perintah artisan serve atau jalankan via Apache XAMPP:

```
php artisan serve
```

- API sekarang dapat diakses di: `http://127.0.0.1:8000/api` atau `http://localhost/craftive/public/api`.

 BAGIAN 2: Skenario Live Demo & Manajemen Waktu (10-15 Menit Presentasi)

Berikut adalah estimasi manajemen waktu dan skenario pembagian topik presentasi kelompok di hadapan Pak Adam agar efisien dan mencakup seluruh aspek penilaian UAS:

WAKTU	DURASI	AKTIVITAS PRESENTASI	DETAIL YANG DITUNJUKKAN & DIJELASKAN
Menit 01 - 02	2 Menit	Pembukaan & Latar Belakang	Perkenalan anggota tim, latar belakang platform Craftive (marketplace kriya premium), tujuan pembuatan aplikasi, serta demo singkat antarmuka website utama.
Menit 03 - 05	3 Menit	Demo Proteksi API & Otentikasi	Pengujian di Postman untuk: 1) API Key (<code>X-API-KEY</code>), 2) HTTP Basic Auth (<code>GET /profile/me</code>), dan 3) Alur login JWT serta registrasi akun baru.
Menit 06 - 09	4 Menit	Demo Alur Transaksi & RBAC	Pengujian di Postman untuk: 1) Penolakan akses CRUD produk (Role Buyer mencoba menambah produk), 2) Pengisian Keranjang (<code>POST /buyer/cart</code>), 3) Checkout transaksi (<code>POST /buyer/checkout</code>), dan 4) Upload bukti bayar Base64.
Menit 10 - 12	3 Menit	Demo Fitur Unggulan & Order	Pengujian fitur Agentic AI Custom Planner (<code>POST /buyer/custom-planner</code>), pengajuan proposal custom ke perajin (<code>POST /custom-proposals</code>), dan persetujuan proposal oleh perajin (<code>PATCH /artisan/proposals/{id}</code>).
Menit 13 - 15	3 Menit	Sesi Tanya Jawab (Q&A)	Menjawab pertanyaan penguji seputar relasi database, stateless token JWT, logika perhitungan AI, kustomisasi middleware, dan keamanan enkripsi password.

 BAGIAN 3: Alur Uji Postman Lengkap (Langkah-demi-Langkah & Payload JSON)

Berikut adalah urutan pengujian API di Postman dari awal sampai akhir beserta contoh payload-nya:

Langkah 1: Pengaturan Environment Postman

- Buka Postman.
- Klik **Environments** di sidebar kiri -> klik **Create Environment**.
- Beri nama environment: `Craftive Local`.
- Tambahkan variabel:
 - `base_url` = `http://localhost/craftive/public/api`
 - `jwt_token` = (biarkan kosong, akan terisi otomatis).
- Pilih environment `Craftive Local` di pojok kanan atas layar Postman.

Langkah 2: Uji Proteksi API Key (Katalog Produk Umum)

- **Tujuan:** Menguji middleware pengaman katalog produk umum dari aktivitas scraping massal.
- **Endpoint:** `GET {{base_url}}/catalog/products`
- **Skenario Uji:**
 1. **Tanpa Header API Key:** Klik **Send** tanpa mengirim header.
 - *Expected Response (401 Unauthorized):*

```
{
  "message": "Unauthorized. Invalid API Key"
}
```

2. **Dengan Header API Key:** Buka tab **Headers**, tambahkan key `X-API-KEY` dengan value `craftive-public-key-2026`. Klik **Send**.
 - *Expected Response (200 OK):*

```
[
  {
    "id": 1,
    "name": "Guci Keramik Kasongan",
    "price": 350000,
    "stock": 15,
    "style": "tradisional"
  }
]
```

Langkah 3: Uji HTTP Basic Auth

- **Tujuan:** Menguji akses profil cepat menggunakan format Base64.
- **Endpoint:** `GET {{base_url}}/profile/me`
- **Skenario Uji:**
 1. Buka request baru di Postman.
 2. Pilih tab **Authorization** -> pilih tipe **Basic Auth**.
 3. Masukkan Username: `admin@craftive.id` dan Password: `password`.
 4. Klik **Send**. (Postman otomatis merubah kredensial menjadi header `Authorization: Basic YWRtaW5AY3JhZnRpdmUuawQ6cGFzc3dvcmQ=`).
 - *Expected Response (200 OK):*

```
{
  "message": "Basic Auth Berhasil!",
  "user": {
    "id": 1,
    "name": "Administrator Craftive",
    "email": "admin@craftive.id",
    "role": "admin"
  }
}
```

Langkah 4: Uji Registrasi Akun & Login JWT

- **Tujuan:** Mendaftarkan pengguna baru dan memperoleh token JWT untuk otorisasi transaksi.

1. Registrasi Akun Baru (POST `{{base_url}}/auth/register`):

- Tab **Body** -> pilih **raw** -> pilih **JSON**.
- *Payload Request:*

```
{
  "name": "Siti Rahayu",
  "email": "siti@craftative.id",
  "password": "password",
  "password_confirmation": "password",
  "role": "buyer"
}
```

- *Expected Response (201 Created)*: mengembalikan data user beserta string token JWT.

2. Login JWT (POST `{{base_url}}/auth/login`):

- *Payload Request:*

```
{
  "email": "siti@craftive.id",
  "password": "password"
}
```

- *Expected Response (200 OK):*

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJIs...",
  "token_type": "bearer",
  "expires_in": 3600,
  "user": {
    "id": 2,
    "name": "Siti Rahayu",
    "email": "siti@craftive.id",
    "role": "buyer"
  }
}
```

- **TIPS Otomatisasi:** Pada tab **Tests** di request login, masukkan kode:

```
var jsonData = pm.response.json();
if(jsonData.access_token) {
    pm.environment.set("jwt_token", jsonData.access_token);
}
```

Token akan otomatis masuk ke variabel environment `{{jwt_token}}` Postman.

Langkah 5: Uji Proteksi RBAC & CRUD Produk

- **Tujuan:** Membuktikan hak akses menu perajin (seller) dilindungi dari pembeli (buyer).
- **Endpoint:** `POST {{base_url}}/artisan/products`
- **Skenario Uji:**
 1. **Uji Role Buyer (Ditolak):** Buka request, pilih tab **Authorization** -> tipe **Bearer Token** -> masukkan variabel `{{jwt_token}}` (yang berisi token milik **buyer** Siti Rahayu). Kirim payload produk.
 - *Expected Response (403 Forbidden):*

```
{
  "message": "Forbidden. You do not have the required role."
}
```

2. **Uji Role Seller/Artisan (Diterima):** Login terlebih dahulu dengan akun perajin (email: `seller@craftive.id`, password: `password`) untuk memperbarui `{{jwt_token}}`. Panggil kembali request tadi dengan payload:
 - *Payload Request:*

```
{
  "category_id": 1,
  "name": "Mangkuk Tanah Liat Kasongan",
  "description": "Mangkuk tanah liat buatan tangan tradisional dengan finishing alami.",
  "price": 45000,
  "stock": 25,
  "style": "Bohemian"
}
```

- *Expected Response (201 Created):*

```
{
  "id": 15,
  "name": "Mangkuk Tanah Liat Kasongan",
  "price": 45000,
  "stock": 25,
  "shop_id": 1
}
```

Langkah 6: Uji Fungsionalitas AI Custom Planner

- **Tujuan:** Mengirimkan spesifikasi kriya buatan tangan pembeli untuk dianalisis biayanya secara otomatis oleh kecerdasan buatan.
- **Endpoint:** `POST {{base_url}}/buyer/custom-planner`
- **Skenario Uji:**
 - Tab **Authorization** -> tipe **Bearer Token** -> isi token **buyer**.
 - *Payload Request:*

```
{
  "specifications": "Meja kayu jati ukiran relief mega mendung ukuran 120x60 cm",
  "materials": "Kayu Jati tua, pelitur cokelat gelap, lem epoxy",
  "budget": 1500000,
  "timeline": 14
}
```

- *Expected Response (200 OK):*

```
{
  "message": "CRAFTIVE AI Custom Planner",
  "planning": {
    "specifications": "Meja kayu jati ukiran relief mega mendung ukuran 120x60 cm",
    "materials": "Kayu Jati tua, pelitur cokelat gelap, lem epoxy",
    "budget": 1500000,
    "timeline_requested": 14,
    "difficulty": "Sangat Rumit",
    "material_cost": 525000,
    "labor_cost": 675000,
    "estimated_price": 1200000,
    "estimated_days": 13,
    "shop_recommendation": "Studio Ukir Jati Abadi",
    "agent_reasoning": "Berdasarkan spesifikasi custom yang Anda masukkan untuk 'Meja kayu jati ukiran"
  }
}
```

Langkah 7: Uji Kirim Proposal Custom ke Perajin

- **Tujuan:** Mengajukan hasil simulasi AI di atas menjadi proposal resmi ke perajin.
- **Endpoint:** `POST {{base_url}}/custom-proposals`
- **Skenario Uji:**
 - *Payload Request:*

```
{
  "craft_type": "Meja Jati Ukir",
  "material": "Kayu Jati, Pelitur",
  "budget": 1500000,
  "deadline_days": 14,
  "description": "Meja kayu jati ukiran relief mega mendung",
  "difficulty": "Sangat Rumit",
  "estimated_days": 13,
  "material_cost": 525000,
  "labor_cost": 675000,
  "shop_recommendation": "Studio Ukir Jati Abadi",
  "agent_reasoning": "Analisis AI menyatakan layak."
}
```

- *Expected Response (201 Created):* Proposal masuk ke DB dengan status `pending` dan ID proposal (misal: `id = 5`).
-

Langkah 8: Uji Persetujuan Proposal (Artisan/Seller Side)

- **Tujuan:** Perajin meninjau proposal masuk dan menyetujuinya, yang otomatis membuat pesanan pembelian baru di database.
- **Endpoint:** `PATCH {{base_url}}/artisan/proposals/5` (ganti `5` dengan ID proposal yang dibuat).
- **Skenario Uji:**
 - Tab **Authorization** -> tipe **Bearer Token** -> gunakan token login milik `artisan`.
 - *Payload Request:*

```
{
  "status": "accepted"
}
```

- *Expected Response (200 OK):*

```
{
  "status": "success",
  "message": "Proposal disetujui. Pesanan transaksi #3 telah dibuat.",
  "proposal": {
    "id": 5,
    "status": "accepted",
    "order_id": 3
  }
}
```

Langkah 9: Uji Tambah ke Keranjang Belanja (Buyer Side)

- **Tujuan:** Menambahkan barang dari katalog ke keranjang belanja pembeli.
- **Endpoint:** `POST {{base_url}}/buyer/cart`
- **Skenario Uji:**
 - Tab **Authorization** -> tipe **Bearer Token** -> gunakan token login milik `buyer`.
 - *Payload Request:*

```
{
  "product_id": 1,
  "qty": 2
}
```

- *Expected Response (201 Created):*

```
{
  "message": "Produk berhasil ditambahkan ke keranjang.",
  "cart": {
    "id": 8,
    "user_id": 2,
    "product_id": 1,
    "qty": 2
  }
}
```

Langkah 10: Uji Checkout Transaksi (Buyer Side)

- **Tujuan:** Memproses semua barang di keranjang belanja menjadi pesanan pembelian resmi.
- **Endpoint:** `POST {{base_url}}/buyer/checkout`
- **Skenario Uji:**
 - Tab **Authorization** -> tipe **Bearer Token** -> gunakan token login milik `buyer`.
 - *Payload Request:*

```
{
  "shipping_address": "Jl. Ketintang No. 156, Gayungan, Surabaya",
  "notes": "Tolong dibungkus bubble wrap yang tebal agar tidak pecah."
}
```

- *Expected Response (200 OK):*

```
{
  "message": "Checkout berhasil dilakukan.",
  "order": {
    "id": 4,
    "buyer_id": 2,
    "total_amount": 700000,
    "status": "pending"
  }
}
```

Langkah 11: Uji Upload Bukti Pembayaran Base64 (Buyer Side)

- **Tujuan:** Mengirim bukti transfer bank berupa string gambar terenkripsi Base64.
- **Endpoint:** `POST {{base_url}}/buyer/payments`
- **Skenario Uji:**
 - Tab **Authorization** -> tipe **Bearer Token** -> gunakan token login milik `buyer`.
 - *Payload Request:*

```
{
  "order_id": 4,
  "proof_image": ""
}
```

- *Expected Response (200 OK):*

```
{
  "message": "Bukti pembayaran berhasil diunggah. Menunggu konfirmasi admin.",
  "payment": {
    "id": 3,
    "order_id": 4,
    "status": "pending"
  }
}
```


Langkah 12: Uji Persetujuan Bukti Bayar & Verifikasi (Admin Side)

- **Tujuan:** Administrator menyetujui transaksi setelah memvalidasi gambar bukti bayar.
- **Endpoint:** `PUT {{base_url}}/admin/orders/4/status` (ganti `4` dengan ID order terkait).
- **Skenario Uji:**
 - Tab **Authorization** -> tipe **Bearer Token** -> gunakan token login milik `admin`.
 - *Payload Request:*

```
{
  "status": "processing"
}
```

- *Expected Response (200 OK):*

```
{
  "message": "Status pesanan berhasil diperbarui.",
  "order": {
    "id": 4,
    "status": "processing"
  }
}
```

Langkah 13: Uji Dokumentasi Interaktif Swagger UI

- **Tujuan:** Memperlihatkan dokumentasi API yang interaktif dan profesional (OpenAPI 3.0) langsung di browser sesuai instruksi Pak Adam.
- **URL Dokumentasi:** `http://localhost/craftive/public/api/documentation` atau `http://localhost/craftive/public/docs` (akan redirect otomatis).
- **Cara Demo:**
 1. Buka browser (Chrome/Edge).
 2. Akses url di atas. Halaman akan menampilkan antarmuka Swagger UI Craftive yang premium.
 3. Tunjukkan bahwa semua rute API Key, Basic Auth, dan JWT terdaftar rapi di sana.
 4. Klik **Authorize** di pojok kanan atas, masukkan parameter auth, lalu uji endpoint secara langsung menggunakan tombol **Try it out**.



BAGIAN 4: Peta Kode Sumber (Direct Files & Line Ranges)

Jika Pak Adam menyuruh Anda membuka kode program di laptop Anda, langsung tunjukkan file-file kunci berikut:

1. File Routing Utama API (`routes/api.php`)

- **Path Absolut:** `c:\xampp1\htdocs\craftive\routes\api.php`
- **Baris 19 - 32:** Rute publik yang dilindungi middleware `api.key`.
- **Baris 41 - 47:** Rute profil dasar cepat menggunakan middleware `auth.basic`.
- **Baris 65 - 151:** Rute transaksional sensitif yang dilindungi middleware `jwt.auth` dan disaring lebih lanjut berdasarkan role (`buyer`, `seller`, `admin`).

2. Kodingan Registrasi & Penerbitan JWT Token

- **Path Absolut:** `c:\xampp1\htdocs\craftive\app\Http\Controllers\Auth\AuthController.php`
- **Baris 16 - 34 (Method `register`):** Validasi data input pendaftaran dan penerbitan token JWT langsung menggunakan guard `api`:

```
$token = Auth::guard('api')->login($user);
```

- **Baris 37 - 51 (Method `login`):** Memvalidasi email dan password terenkripsi. Menggunakan fungsi `attempt()` untuk mencocokkan kredensial dan menerbitkan token:

```
if (! $token = Auth::guard('api')->attempt($credentials)) { ... }
```

3. Kodingan Middleware API Key

- **Path Absolut:** `c:\xampp1\htdocs\craftive\app\Http\Middleware\ApiKeyMiddleware.php`
- **Baris 10 - 17 (Method `handle`):** Membaca header request `X-API-KEY` dan membandingkannya dengan kunci statis `craftive-public-key-2026`:

```
$apiKey = $request->header('X-API-KEY');  
if ($apiKey !== 'craftive-public-key-2026') {  
    return response()->json(['message' => 'Unauthorized. Invalid API Key'], 401);  
}
```

4. Kodingan Middleware Role Otorisasi (RBAC)

- **Path Absolut:** `c:\xampp1\htdocs\craftive\app\Http\Middleware\RoleMiddleware.php`
- **Baris 11 - 18 (Method `handle`):** Mengekstrak data user terotentikasi, lalu memeriksa apakah role-nya sesuai dengan parameter middleware variadic:

```
public function handle(Request $request, Closure $next, ...$roles): Response {  
    $user = Auth::user();  
    if (!$user || !in_array($user->role, $roles)) {  
        return response()->json(['message' => 'Forbidden...'], 403);  
    }  
    return $next($request);  
}
```

5. Kodingan Alur Custom Proposal (Logic Pembuatan Order Otomatis)

- **Path Absolut:** `c:\xampp1\htdocs\craftive\app\Http\Controllers\Buyer\CustomProposalController.php`
- **Baris 95 - 164 (Method `accept`):**
 1. Mengecek kecocokan perajin (seller) penerima proposal.
 2. Membuat entri produk kustom baru di tabel `products` dengan status `is_active = false` (agar tersembunyi dari katalog umum).

3. Membuat entri pesanan baru di tabel `orders` dengan total harga sesuai budget yang disetujui.
4. Membuat detail item pesanan di tabel `order_items` yang merujuk pada produk kustom tadi.
5. Memperbarui status proposal kustom menjadi `accepted`.

6. Kodingan Kalkulator Heuristik AI Planner

- **Path Absolut:** `c:\xampp1\htdocs\craftive\app\Http\Controllers\AiRecommendationController.php`
- **Baris 63 - 128 (Method `planCustomKriya`):** Logika heuristik backend untuk mensimulasikan biaya material, tarif jasa tukang berdasarkan timeline/budget, durasi kerja optimal, serta penalaran penjelasan agen AI.

BAGIAN 5: Alur Sambungan Fitur & Lokasi Kode (Bagaimana Frontend Memanggil Backend)

Di bawah ini adalah penjelasan detail mengenai **bagaimana fitur utama di website menyambung ke API backend**, teknologi yang digunakan, serta **lokasi file dan baris kodenya** secara presisi:

1. Fitur Katalog & Pencarian Produk

- **Pake Apa?** Fungsi JavaScript `fetch()` (menggunakan framework Alpine.js) dengan menyisipkan header `X-API-KEY`.
- **Bagaimana Alurnya?**
 1. **Frontend (Tampilan):** Pengguna masuk ke halaman katalog, atau mengetik kata kunci di kolom pencarian.
 - **Lokasi Kode:** `resources/views/pages/products.blade.php` pada method `loadProducts()` (Baris 274 - 305). JavaScript memicu `fetch('/api/products?search=...')` dengan membawa header `'X-API-KEY': 'craftive-public-key-2026'`.
 2. **Backend Routing:** Rute menerima request ini dan meneruskannya ke filter keamanan.
 - **Lokasi Kode:** `routes/api.php` pada Baris 21 & 30 (`Route::get('/products', ...)`).
 3. **Backend Middleware (Filter):** Mengamankan katalog dari bot/scraping luar.
 - **Lokasi Kode:** `app/Http/Middleware/ApiKeyMiddleware.php` pada fungsi `handle()` (Baris 10 - 17). Memastikan isi header `X-API-KEY` bernilai cocok.
 4. **Backend Controller (Logika):** Menarik data dari database dan mengembalikannya sebagai JSON.
 - **Lokasi Kode:** `app/Http/Controllers/Public/ProductController.php` pada method `index()`.
 5. **Frontend Render:** JavaScript menerima data JSON, memasukannya ke variabel Alpine.js (`this.products`), lalu merender kartu produk ke layar secara reaktif menggunakan direktif `x-for` (Baris 133 - 208).

2. Fitur Kelola Keranjang Belanja (Cart)

- **Pake Apa?** Fungsi JavaScript pembungkus `window.apiFetch()` (yang menyisipkan token JWT Bearer secara otomatis di header) untuk mengirim request `POST` / `DELETE` berformat JSON.
- **Bagaimana Alurnya?**
 1. **Frontend (Tampilan):** Pembeli mengklik tombol **"Add"** pada salah satu produk kriya.
 - **Lokasi Kode:** `resources/views/pages/products.blade.php` pada method `addToCart(productId)` (Baris 336 - 368). Mengirim `POST` ke `/api/buyer/cart` membawa JSON body `{"product_id": 1, "qty": 1}`.
 2. **Otomatisasi Token JWT:**

- **Lokasi Kode:** [resources/views/layouts/app.blade.php](#) pada helper JS `window.apiFetch`. Fungsi ini membaca token JWT pembeli dari `localStorage` browser dan menyisipkannya ke header `Authorization: Bearer <token_jwt>`.

3. Backend Routing & Middleware (Otorisasi):

- **Lokasi Kode:** [routes/api.php](#) pada Baris 97 - 99. Dilindungi middleware `'jwt.auth'` (memastikan login aktif) dan middleware `'role:buyer'` (memastikan peran adalah pembeli).
- **Role Filter:** [app/Http/Middleware/RoleMiddleware.php](#) pada fungsi `handle()` (Baris 11 - 18).

4. Backend Controller (Logika):

- **Lokasi Kode:** [app/Http/Controllers/Buyer/CartController.php](#) pada method `store()` untuk menyimpan item belanja ke tabel `carts`.

3. Fitur Agentic AI Custom Planner

- **Pake Apa?** Mengirimkan input teks spesifikasi material, budget, dan timeline pembeli menggunakan request `POST` JSON terproteksi JWT.
- **Bagaimana Alurnya?**
 1. **Frontend (Tampilan):** Pembeli mengisi form simulasi kriya kustom di dashboard pembeli, lalu menekan "Mulai Simulasi AI".
 - **Lokasi Kode:** [resources/views/user/dashboard.blade.php](#) pada fungsi JS `submitAiPlanner()`.
 2. **Backend Routing:**
 - **Lokasi Kode:** [routes/api.php](#) pada Baris 69 & 103 (`Route::post('/buyer/custom-planner', ...)`).
 3. **Backend Controller (AI Logic Heuristik):**
 - **Lokasi Kode:** [app/Http/Controllers/AiRecommendationController.php](#) pada fungsi `planCustomKriya()` (Baris 63 - 128). Backend menghitung alokasi budget (35% material, 45% tarif perajin), menguji timeline/kesulitan kriya, merekomendasikan sanggar perajin yang cocok, menyimpan riwayat analisis ke tabel `ai_recommendations`, dan mengembalikan data JSON analisis ke browser.

4. Fitur Proposal Kustom & Pembuatan Order Otomatis (Strict Order Workflow)

- **Pake Apa?** Request `PATCH` berisi persetujuan status proposal dari perajin (seller) untuk memicu database transaction yang membuat pesanan baru secara otomatis.
- **Bagaimana Alurnya?**
 1. **Frontend (Tampilan):** Perajin membuka kotak masuk proposal kustom di dashboard-nya, lalu mengklik tombol "Setujui Proposal" (status: `accepted`).
 - **Lokasi Kode:** [resources/views/seller/dashboard.blade.php](#) pada fungsi JS `acceptProposal(id)`.
 2. **Backend Routing & Middleware:**
 - **Lokasi Kode:** [routes/api.php](#) pada Baris 122 (`Route::patch('/proposals/{id}', ...)`). Dilindungi middleware `'role:seller'`.
 3. **Backend Controller (Logika Order Otomatis):**
 - **Lokasi Kode:** [app/Http/Controllers/Buyer/CustomProposalController.php](#) pada fungsi `accept()` (Baris 95 - 164).
 - **Logika Bisnis:**
 - Membuat produk baru secara dinamis di database dengan status `is_active = false` agar tersembunyi dari katalog umum.
 - Membuat pesanan baru di tabel `orders` dengan total harga sesuai budget yang disetujui.

- Membuat detail item pesanan di tabel `order_items` yang merujuk pada produk kustom tadi.
- Memperbarui status proposal kustom di tabel `custom_proposals` menjadi `accepted`.

? BAGIAN 6: Bank Pertanyaan Ujian UAS & Cara Menjawabnya (Akademik Premium)

Berikut adalah prediksi pertanyaan tajam yang biasa diajukan oleh Pak Adam saat demo proyek API beserta jawaban akademis yang terstruktur:

💡 Pertanyaan 1: "Jelaskan bagaimana middleware `jwt.auth` memvalidasi token secara stateless!"

Cara Menjawab:

"REST API dirancang secara stateless, artinya server tidak menyimpan sesi login user di memori lokal (seperti file session PHP biasa). Setiap kali request masuk ke rute yang dilindungi, middleware `jwt.auth` akan membaca string token Bearer dari header `Authorization`. Token tersebut kemudian didekode menggunakan kunci rahasia aplikasi (`JWT_SECRET`) yang ada di file `.env`. Middleware memeriksa tanda tangan (signature) token untuk memastikan token tersebut sah, belum kedaluwarsa, dan diterbitkan oleh server kami. Jika valid, middleware mengambil ID user dari payload token (`sub` claim) dan mengikat data pengguna tersebut ke instance Request saat itu."

💡 Pertanyaan 2: "Mengapa Anda menggunakan status code `201 Created` untuk registrasi sedangkan login menggunakan `200 OK`?"

Cara Menjawab:

"Sesuai dengan spesifikasi standar arsitektur RESTful API (RFC 7231), status code `201 Created` digunakan secara khusus ketika suatu request berhasil membuat sumber daya (resource) baru yang permanen di server (dalam kasus ini, membuat data pengguna baru di tabel `users` database MySQL). Sementara itu, login bukanlah proses pembuatan resource baru melainkan proses validasi kredensial dan pengambilan token akses sementara, sehingga cukup menggunakan status code standard `200 OK`."

💡 Pertanyaan 3: "Bagaimana cara kerja middleware Role/RBAC Anda jika parameter filternya lebih dari satu?"

Cara Menjawab:

"Pada file `routes/api.php`, kami mendaftarkan middleware role dengan parameter variadic, misalnya `middleware('role:seller,admin')`. Laravel mem-parsing nilai parameter tersebut menjadi array menggunakan sintaks variadic PHP `...$roles` pada method middleware. Di dalam `RoleMiddleware`, kami memvalidasi role user terotentikasi menggunakan fungsi bawaan PHP `in_array($user->role, $roles)`. Dengan cara ini, rute tersebut dapat diakses oleh pengguna yang memiliki peran sebagai penjual (seller) maupun administrator."

💡 Pertanyaan 4: "Bagaimana integrasi AI Custom Planner dilakukan di backend Laravel?"

Cara Menjawab:

"Integrasi AI Custom Planner berjalan di controller `AiRecommendationController`. Controller menerima data spesifikasi produk, material, target budget, dan timeline pengerjaan yang dimasukkan oleh pembeli. Backend kemudian melakukan simulasi perhitungan biaya secara dinamis menggunakan aturan heuristik terprogram: alokasi 35% budget untuk estimasi biaya bahan baku, 45% untuk biaya jasa perajin, durasi pengerjaan logis, serta penentuan tingkat kesulitan. Hasil analisis terstruktur ini disimpan ke tabel `ai_recommendations` sebagai histori, lalu dikembalikan sebagai respon JSON yang memuat saran studio perajin terdekat dan penalaran keputusan agen AI."

💡 Pertanyaan 5: "Bagaimana Anda mengamankan password user di database MySQL?"

Cara Menjawab:

"Password pengguna tidak pernah disimpan dalam bentuk teks biasa (plain text). Pada method `register` di `AuthController`, kami mengenkripsi password menggunakan fungsi `Hash::make($request->password)`. Fungsi ini memanfaatkan algoritma hashing `bcrypt` bawaan Laravel yang dilengkapi dengan salt unik secara otomatis. Algoritma `bcrypt` dirancang lambat secara komputasi untuk meredam serangan brute force, sehingga data password pengguna tetap aman meskipun database mengalami kebocoran."

💡 Pertanyaan 6: "Jelaskan relasi tabel antara `orders`, `order_items`, dan `products` pada database Anda!"

Cara Menjawab:

"Relasi tabel dirancang secara relasional untuk menjaga integritas data transaksi:

1. Tabel `orders` memiliki relasi **One-to-Many** ke tabel `order_items`, di mana satu pesanan dapat memiliki banyak item barang belanjaan (dihubungkan via foreign key `order_id`).
2. Tabel `products` memiliki relasi **One-to-Many** ke tabel `order_items` (dihubungkan via foreign key `product_id`), yang berarti sebuah produk kriya dapat dipesan di banyak baris item transaksi.
3. Di tingkat model Eloquent Laravel, kami mendefinisikan relasi tersebut menggunakan fungsi `hasMany()` pada model `Order`, `belongsTo()` pada model `OrderItem` ke model `Order` dan `Product`, serta `belongsTo()` dari model `Product` ke model `Shop`."

💡 Pertanyaan 7: "Mengapa Anda menggunakan tipe data JSON pada kolom `images` di tabel `products`?"

Cara Menjawab:

"Kami menggunakan tipe data JSON pada kolom `images` untuk mendukung penyimpanan multi-gambar produk tanpa memerlukan tabel tambahan (seperti tabel `product_images`). Laravel Eloquent mempermudah manipulasi kolom ini dengan fitur casting atribut `protected $casts = ['images' => 'array']` pada model `Product`. Dengan casting ini, Laravel otomatis mengubah array PHP menjadi string JSON saat disimpan ke database MySQL, dan mengubahnya kembali menjadi array PHP saat data dibaca dari database."

💡 Pertanyaan 8: "Apa fungsi dari perintah `php artisan key:generate` dan `php artisan jwt:secret`?"

Cara Menjawab:

"`php artisan key:generate` digunakan untuk mengisi nilai kunci enkripsi utama aplikasi (`APP_KEY`) di file `.env`. Kunci ini digunakan oleh Laravel untuk mengamankan data terenkripsi seperti session web dan cookie. Sementara itu, `php artisan jwt:secret` digunakan khusus oleh package `tymon/jwt-auth` untuk menghasilkan kunci rahasia signature JWT (`JWT_SECRET`) di file `.env`. Kunci ini digunakan untuk menandatangani (sign) token JWT yang diterbitkan sehingga pihak luar tidak dapat memalsukan token tersebut tanpa mengetahui kunci rahasianya."

💡 Pertanyaan 9: "Bagaimana cara kerja upload bukti transfer dalam format Base64 di API Anda?"

Cara Menjawab:

"Pembeli mengirimkan gambar bukti transfer bank dalam bentuk string terenkripsi Base64 melalui payload JSON `proof_image`. Di controller `PaymentController`, kami memvalidasi format string Base64 tersebut. Jika valid, string Base64 didekode menjadi data biner mentah menggunakan fungsi PHP `base64_decode()`. File gambar biner tersebut kemudian disimpan ke media penyimpanan lokal server menggunakan Facade `Storage::put()`, dan path file gambarnya disimpan ke kolom `proof_image` di tabel `payments`. Pendekatan ini mempermudah transmisi gambar melalui protokol REST API murni tanpa perlu menggunakan format request multi-part form data."

💡 Pertanyaan 10: "Bagaimana jika perajin (seller) menolak proposal kustom pembeli? Apa yang terjadi di database?"

Cara Menjawab:

"Jika perajin menolak proposal kustom melalui endpoint `reject` (`PATCH /api/artisan/proposals/{id}` dengan status `rejected`), sistem hanya memperbarui kolom `status` pada baris data proposal di tabel `custom_proposals` menjadi `rejected`. Berbeda dengan status disetujui (`accepted`), status ditolak tidak akan memicu pembuatan produk kustom baru di tabel `products` maupun pembuatan transaksi baru di tabel `orders` dan `order_items`."

💡 Pertanyaan 11: "Mengapa Anda mendefinisikan alias route seperti `/api/artisan/...` sedangkan di database role-nya bernama `seller`?"

Cara Menjawab:

"Penggunaan nama `/api/artisan/` pada route prefix bertujuan untuk menyesuaikan dengan penamaan domain bisnis aplikasi Craftive yang bertema perajin lokal (`artisan`). Namun, untuk menjaga konsistensi dengan standar otorisasi backend dan database, kami tetap menggunakan nama peran `seller` di kolom database. Kami menyelarkannya di file `routes/api.php` dengan membungkus route prefix `artisan` di dalam middleware `role:seller,admin`, sehingga secara logika bisnis rute tersebut dikhususkan bagi perajin."

💡 Pertanyaan 12: "Bagaimana Anda memastikan transaksi checkout aman dan tidak mengalami kebocoran stock jika terjadi error di tengah proses?"

Cara Menjawab:

"Untuk menjamin integritas transaksi di database, proses checkout di `OrderController` dibungkus menggunakan

fitur **Database Transactions** bawaan Laravel (`DB::beginTransaction()` dan `DB::commit()`). Jika proses pengurangan stok produk atau pembuatan entri order items mengalami kegagalan di tengah jalan, Laravel akan langsung memicu `DB::rollBack()` . Hal ini membatalkan seluruh perubahan database yang sempat berjalan sebelumnya, sehingga database kembali ke keadaan semula dan mencegah terjadinya data transaksi menggantung atau salah perhitungan stok."

💡 **Pertanyaan 13: "Jelaskan perbedaan antara autentikasi JWT, API Key, dan Basic Auth pada proyek Anda!"**

Cara Menjawab:

"Kami menerapkan konsep autentikasi tiga lapis dengan karakteristik yang berbeda sesuai kebutuhan keamanan:

1. **Basic Auth:** Menggunakan header `Authorization: Basic [Base64(email:password)]` . Digunakan untuk autentikasi cepat saat mengakses data profil user (`/api/profile/me`).
2. **API Key:** Menggunakan header custom `X-API-KEY: craftive-public-key-2026` . Bersifat statis dan digunakan untuk memproteksi endpoint katalog produk publik (`/api/catalog/products`) dari scraping data massal oleh bot luar.
3. **JWT Bearer Token:** Menggunakan token terenkripsi dinamis stateless yang diterbitkan setelah proses login berhasil. Digunakan untuk mengamankan seluruh transaksi sensitif seperti keranjang, checkout, dan pengelolaan produk perajin."

💡 **Pertanyaan 14: "Mengapa Anda memilih framework Laravel untuk membuat REST API ini daripada Node.js Express?"**

Cara Menjawab:

"Kami memilih Laravel karena menyediakan ekosistem pengembangan yang sangat lengkap dan mempercepat pembuatan aplikasi (Rapid Application Development). Bawaan Laravel sudah menyediakan sistem ORM Eloquent yang kuat untuk relasi database, fitur migrasi database yang rapi, middleware siap pakai, serta integrasi yang mudah dengan package pihak ketiga seperti `jwt-auth` untuk otentikasi stateless. Hal ini memungkinkan kami membangun REST API yang aman dan terstruktur dalam waktu singkat."

💡 **Pertanyaan 15: "Bagaimana cara menangani kedaluwarsa token JWT di aplikasi Anda?"**

Cara Menjawab:

"Token JWT memiliki masa berlaku terbatas (default 60 menit) yang diatur melalui konfigurasi `tTL` (Time to Live). Jika token kedaluwarsa, server akan mengembalikan status code `401 Unauthorized` dengan pesan token expired. Untuk menjaga kenyamanan pengguna agar tidak perlu login ulang berulang kali, kami menyediakan endpoint `POST /api/auth/refresh` . Frontend dapat mengirimkan token yang hampir kedaluwarsa untuk ditukarkan dengan token baru yang segar melalui fungsi `Auth::guard('api')->refresh()` ."

🔧 **BAGIAN 6: Panduan Troubleshooting Cepat Saat Live Demo**

Jika terjadi kendala teknis atau error secara mendadak di hadapan Pak Adam, jangan panik! Gunakan panduan berikut untuk memperbaikinya dengan cepat:

Error 1: "Database Connection Refused" / "SQLSTATE[HY000] [2002]"

- **Penyebab:** MySQL Server di XAMPP belum dijalankan, atau port MySQL berubah.
- **Cara Mengatasi:**
 1. Buka **XAMPP Control Panel**.
 2. Pastikan modul **Apache** dan **MySQL** dalam kondisi berwarna hijau (Running).
 3. Jika port MySQL Anda bukan default (3306), buka file `.env` lalu sesuaikan nilai variabel `DB_PORT` dengan port MySQL yang sedang berjalan di XAMPP Anda.

Error 2: "401 Unauthorized - Invalid API Key" di Postman

- **Penyebab:** Lupa menambahkan header `X-API-KEY` pada request katalog produk publik.
- **Cara Mengatasi:**
 1. Buka request Anda di Postman.
 2. Masuk ke tab **Headers** (di bawah kolom URL).
 3. Tambahkan baris baru: Key = `X-API-KEY` , Value = `craftive-public-key-2026` .
 4. Klik **Send** kembali.

Error 3: "401 Unauthorized" atau "Token Expired" saat Mengakses Menu Transaksi

- **Penyebab:** Token JWT Bearer Anda sudah kedaluwarsa (lebih dari 60 menit) atau Anda belum melakukan login ulang setelah database di-refresh.
- **Cara Mengatasi:**
 1. Lakukan request **Login** kembali dengan akun user terkait (POST `/api/auth/login`).
 2. Ambil nilai `access_token` terbaru dari respon JSON.
 3. Buka request transaksi Anda, masuk ke tab **Authorization**, pilih tipe **Bearer Token**, dan tempelkan token baru tersebut ke kolom yang disediakan.

Error 4: "403 Forbidden - You do not have the required role"

- **Penyebab:** Anda menggunakan token pembeli (buyer) untuk memanggil rute khusus perajin (seller) seperti `/api/artisan/products` .
- **Cara Mengatasi:**
 1. Jalankan request login menggunakan akun perajin (email: `seller@craftive.id` , password: `password`).
 2. Salin token JWT perajin yang diterbitkan ke request Anda.
 3. Kirim kembali request Anda.

Error 5: "422 Unprocessable Entity - Validation Errors"

- **Penyebab:** Payload JSON yang Anda kirimkan tidak memenuhi kriteria validasi controller Laravel (misalnya tipe data salah atau ada kolom wajib yang kosong).
- **Cara Mengatasi:**
 1. Periksa bagian respon JSON error untuk melihat kolom mana yang tidak memenuhi syarat validasi.
 2. Sesuaikan payload request Anda di tab **Body JSON** Postman sesuai contoh payload yang ada di panduan ini.